Programación Lógica

Unidad VI: Interfaz gráfica

PRESENTADO POR:

MSc. Claudia E. Martínez Darce

León, 23 y 25 marzo 2026

2. Biblioteca XPCE- Interface gráfica

XPCE es un sistema orientado a objeto, se distribuye como una biblioteca sobre el sistema Prolog de hospedaje, para usarlo en **SWI-Prolog** debe cargarse con los predicados básicos o explícitamente con el siguiente predicado en cualquier parte del archivo (preferible al principio):

```
?- use_module(library(pce)).
true.
```

Es un enlace simbólico a pl, la versión indicada es 5.1 y el nivel de parche es 0, generalmente.

Una vez cargada la librería, se dispone de una serie de predicados para poder crear **botones, ventanas**, y muchos otros **objetos**.

2. Predicados básicos de XPCE

Abordaremos **cuatro** predicados básicos que se utilizan para controlar XPCE desde Prolog, los cuales se asignan a las funciones básicas de la máquina virtual:

1. Crear objetos **(new).**
2. Manipular o recibir mensajes de objetos **(send).**
3. Consultar o recibir mensajes de objetos **(get).**
4. Destruir o eliminar memoria de objetos **(free).**

En el mundo de **XPCE** hay objetos que representan una posición en un plano bidimensional, así como una ventana en pantalla, cada objeto tiene un estado, podremos llamar **métodos** de dichos objetos (pasando los parámetros correspondientes) o llamándolos para recibir algún valor.

Referencias.

Prolog usa principalmente dos tipos de referencias:

-Una que seria mediante las **variables** de prolog (como Variable, Cantidad, ...),

-Otra forma es definir **referencias** con nombre, las cuales una vez definidas no podremos volver a crear otra referencia para dicho nombre, ya que nos dará error en ejecución.

Estas últimas referencias resultan interesantes para acceder de forma global a un mismo objeto, sin tener que pasar ningún tipo de parámetro. Estas variables se crean usando el operador especial @, cualquier nombre que empiece por @ será una variable asociada a ese nombre que le demos (por ejemplo, **@demo, @exa**, etc).

a) Creación de objetos

Predicado	Descripción
new(?Reference,+NewTerm).	Crea un objeto en el mundo de XPCE y le asigna la referencia dada o unifica el primer argumento con XPCE. El predicado recoge dos parámetros: - el primero recogería la referencia que se le asigna al nuevo objeto, ya que **new** se usa para crear objeto, - y el segundo parámetro le indicará el objeto que se quiere crear.

Las referencias se usan para saber a que objeto nos referimos, así pues cada objeto que creemos deberá tener su propia referencia, ya que después toda memoria que reservemos con **new**, será conveniente liberarla con **free**.

a) Creación de objetos

1

```
?- new(P, point(10,20)).
P = @13135591/point.
```

Crea una instancia de la clase **point** a partir de los argumento 10, 20 que son un punto en la ubicación de las coordenadas X, Y

2

```
?- new(@demo,dialog('Mi Ventana')).
true.
```

Crea un objeto de **dialog** (ventana para mostrar controladores: botones, cuadros de textos, listas y más), las referencias se representan usando el operador @/1. En este ejemplo el título de la ventana de diálogo es ‘Mi Ventana’.

3

```
?- new(@d,size(100,5)).
true.
```

Crea una instancia de la clase **size** a partir de los argumento de ancho 100 y alto 5, el nombre del objeto será @d que es la referencia.

b) Manipular de objetos

Predicado	Descripción
send(?Receiver,+Selector(…Args…).	Permite manipular el estado de un objeto. El primer parámetro del predicado es una referencia al objeto al deseemos enviarle un mensaje; El segundo parámetro indicará el método al que queremos invocar, lo cual indicaremos junto a los argumentos que deseemos.

b) Manipular de objetos

1

```
?- send(@14063723,x(15)).
```

Invoca el método **x** del punto del objeto, instancia a la variable x el correspondiente punto del objeto con el argumento del valor 15.

2

```
?- send(@demo,append(text_item(nombre))).
true.
```

Llamada a la clase **append** de **dialog,** agrega un componente a la ventana de diálogo, por medio del termino *text_item*(name)

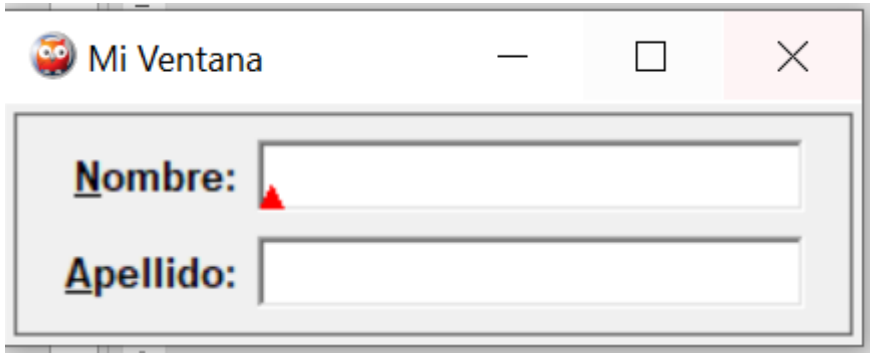
3

```
?- send(@demo,open).
true.
```

Permite abrir la ventana de diálogo creada, por medio de la función **open** nombrando a la referencia.

b) Manipular de objetos

```
?- use_module(library(pce)).
true.

?- new(P, point(10,20)).
P = @13972252/point.

?- new(@demo,dialog('Mi Ventana')).
true.

?- send(@13972252,x(15)).
true.

?- send(@demo,append(text_item(nombre))).
true.

?- send(@demo,append(text_item(apellido))).
true.

?- send(@demo,open).
true.
```



c) Consultar estado de objetos

Predicado	Descripción
get(?Receiver,+Selector(+Arg…),-Result).	Es usado para obtener información del estado del objeto. Los dos primeros parámetros tienen el mismo significado que para **send,** sin embargo el último parámetro sirve para recoger el valor que nos devuelva el **método**. Generalmente este valor de retorno es un objeto de referencia, excepto para los objetos de nombre XPCE que se devuelven como un átomo de Prolog.

c) Consultar estado de objetos

1

```
?- get(@13972252,y,Y).
Y = 20.
```

Permite obtener el valor de la variable Y.

2

```
?- get(@demo,display,D).
D = @display/display.
```

Devuelve el objeto de visualización en el que se muestra @demo. Esta es la referencia a un objeto de la clase **display** que representa su pantalla.

3

```
?- get(@13972252,distance(point(100,100)),Distance).
Distance = 116.
```

Muestra de nuevo la creación de objeto a partir de los argumentos a **send/2** y **get/3** y también muestra el valor devuelto, no es necesario sea una variable de instancia directa del objeto, el valor devuelto representa la distancia (redondeada) entre @13972252 y el punto (100,100)

c) Consultar estado de objetos

1

```
?- get(@display,size,Size),
|     get(Size,width,W),
|     get(Size,height,H).
Size = @14069172/size,
W = 3200,
H = 1080.
```

En este ejemplo **get/3** devuelve objetos por su referencia, estas referencias se puede usar para otras consultas, aquí específicamente se calcula el ancho y alto de su pantalla.

2

```
?- get(@demo,member(nombre),TextItem),get(TextItem,selection,Text).
TextItem = @14154957/text_item,
Text = 'Claudia.'.
```

El primer get/3 solicita con **member** el cuadro de diálogo con el **nombre**, este devuelve la referencia de objeto de elemento del texto enviado al cuadro de diálogo.
El segundo get/3 obtiene por medio de **select** el texto escrito por el usuario.

d) Destruir objetos

Predicado	Descripción
free(?Reference).	Librera la memoria asociada al objeto que se le indica en el primer parámetro. Elimina el objeto de la base de objetos XPCE.

```
?- free(@13972252).
true.

?- free(@demo).
true.

?- free(@display).
false.
```

El segundo **free/1** no solo elimina el objeto de la ventana de diálogo de la base de objetos XPCE sino que también elimina la ventana asociada de la pantalla.

El último ejemplo da **false** porque ciertos objetos del sistema han sido protegidos contra la liberación.

d) Destruir objetos

Cuando usemos las **variables** con nombre deberemos llevar especial cuidado ya que deberemos **liberarlas** usando **free** antes de que finalice la evaluación del predicado, ya que si volvemos a realizar la misma consulta, y volvemos a crear objetos usando esas mismas variables, genera **error** y no dejará **ejecutar**.

¿Cómo funciona?

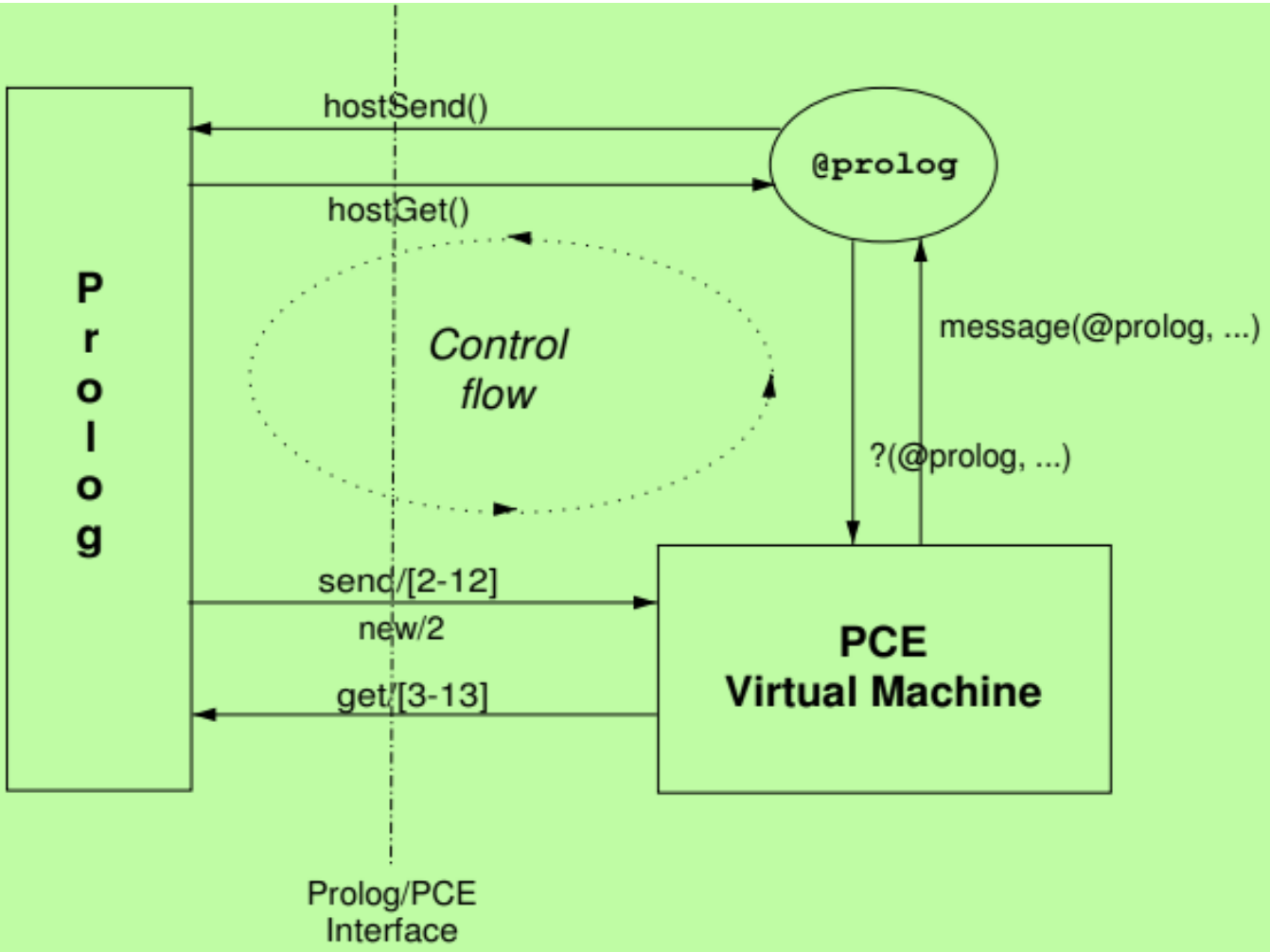
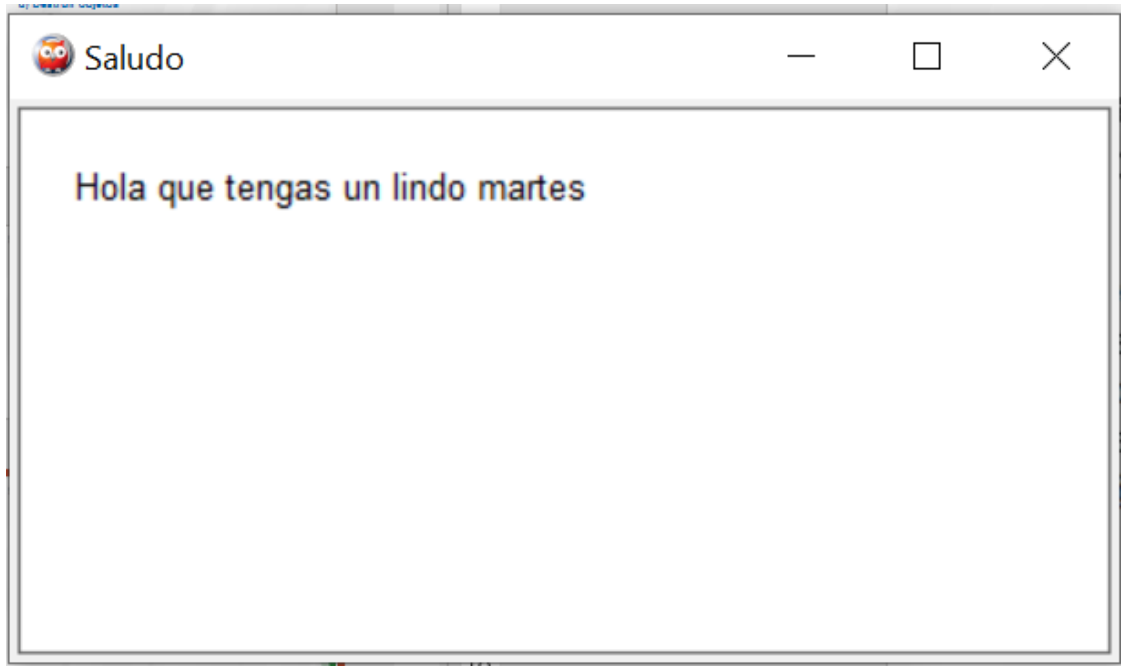


Figure 2.2: Data and Control flow in PCE/Prolog

Ejemplo1:

Crea un programa que envíe un saludo en día de hoy dentro de una ventana de diálogo.

```
?- new(P,picture('Saludo')),
|    send(P, display, text('Hola que tengas un lindo martes'),
|    point(20,20)),
|    send(P,open).
P = @294361905955/picture.
```



Agregar los comandos necesarios para eliminar de memoria las referencias a los objetos creados.

Intenta:

Programa que crea una ventana que se nombre ‘Nicaragua’ permita ingresar el nombre de un departamento y luego lo capture para mostrar en pantalla.

Programa que crea una ventana que muestre un formulario en donde se permita ingresar: *Curso, Carrera, Año.* Intenta capturar el texto ingresado y mostrar en pantalla.

¡Muchas Gracias!

¿Preguntas?